Science of Computer Programming 239 (2025) 103181



Contents lists available at ScienceDirect

Science of Computer Programming

journal homepage: www.elsevier.com/locate/scico



Original Software Publication



Microservices simulator: An object-oriented framework for transactional causal consistency

Pedro Pereira, António Rito Silva *

INESC-ID, Instituto Superior Técnico, University of Lisbon, Rua Alves Redol 9, 1000-029 Lisbon, Portugal

ARTICLE INFO

Keywords:
Microservices
Simulator
Transactional causal consistency

ABSTRACT

The development of microservice systems is complex due to the impact consistency problems have on the business logic design. On the other hand, the implementation of a microservice requires extensive use of middleware technology, which delays the test and identification of the cases where consistency problems can occur. We introduce the *Microservices Simulator* object-oriented framework that supports the rapid prototyping of a microservice system designed according to the Transactional Causal Consistency model.

1. Motivation and significance

Microservice systems [1] are modeled following the *Domain-Driven Design* (DDD) [2] concepts of *Aggregate* and *Upstream-Downstream* relationships between development teams. In the development of microservice systems, the implementation of the system business logic and the coordination of the services execution is done later in the development process, due to the complexity of middleware technology and the execution in a distributed context. To address this problem, the Microservices Simulator [3] allows rapid specification of a microservice system and its testing and experimentation according to *Transactional Causal Consistency* (TCC) [4], following DDD (see Table 1 for the code metadata).

2. Software description

The simulator decouples the specification of the microservice business logic from the definition of TCC behavior. Then system can be exercised by implementing test cases.

2.1. Software architecture

Fig. 1 presents a decomposition view of the simulator that includes an extensive application, *quizzes*, as an example of usage.

The *domain* module contains the common software elements that are reused to define a microservice. It comprises the concepts of *aggregate* and *event*. Aggregates define the unit of concurrency control where atomic transactions are executed. Events represent interactions between upstream and downstream aggregates. Upstream aggregates publish events that are subscribed by interested downstream aggregates. The domain software elements of the architecture are colored red.

* Corresponding author.
E-mail addresses: pedro.l.pereira@tecnico.ulisboa.pt (P. Pereira), rito.silva@tecnico.ulisboa.pt (A. Rito Silva).

https://doi.org/10.1016/j.scico.2024.103181
Received 26 October 2023; Received in revised form 7 June 2024; Accepted 24 July 2024

Available online 31 July 2024
0167-6423/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Table 1
Code metadata.

Nr.	code metadata description	
C1	current code version	v1.1.4
C2	permanent link to code/repository used for this code version	https://github.com/socialsoftware/business-logic-consistency-models
C3	permanent link to reproducible capsule	https://zenodo.org/doi/10.5281/zenodo.7854924
C4	legal code license	MIT license
C5	code versioning system used	git
C6	software code languages, tools, and services used	Java, spring boot, JPA
C7	compilation requirements, operating environments and dependencies	docker compose
C8	developer documentation	https://github.com/socialsoftware/business-logic-consistency-models#readme
C9	support email for questions	rito.silva@tecnico.ulisboa.pt

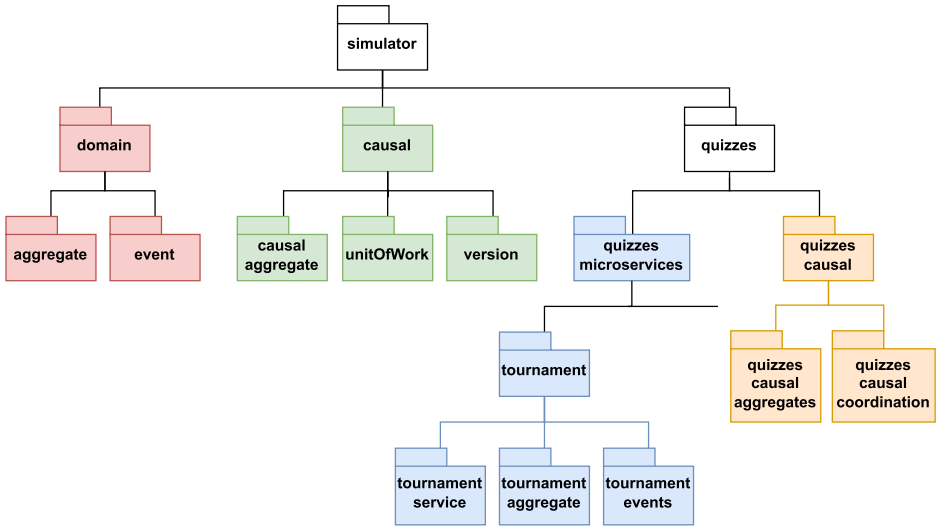


Fig. 1. Simulator Decomposition View.

The *causal* module is responsible for the TCC behavior of the simulator. The *causal aggregate* module adds TCC semantics to the aggregate. The *unitOfWork* module implements the *Unit of Work* pattern [5] for TCC, which defines the transactional behavior associated with orchestrated execution aggregates' atomic services. Finally, the *version* module supports aggregate versioning, which is used to implement TCC. The causal software elements are colored green.

The *quizzes* module contains an example of a microservice system implemented using the simulator. It has two submodules: *quizzes microservices* and *quizzes causal*. The former contains the microservices business logic (only *tournament* is represented) that extends from the *domain* module. Each microservice has three submodules, where the *services* contain the atomic services that constitute their API. The *quizzes microservices* software elements are colored blue. The *quizzes causal* module extends the microservices with causal behavior by using the *causal* module. In particular, *quizzes causal coordination* module contains the coordinated execution of microservices services using TCC. The *quizzes causal* software elements are colored orange.

2.2. Software functionalities

To implement a microservice system in the simulator, it is necessary to extend and use classes that are in the *domain* and *causal* modules.

2.2.1. Define microservices

Fig. 2 presents the classes that need to be extended and the methods to redefine. Colors are used to indicate to which modules the classes belong, as represented in Fig. 1.

There are two main steps: (1) define the microservice business logic independently of a particular transactional model (blue); (2) enrich microservices with the TCC behavior (orange). Both specifications are done according to the description in [3]. Note that *Aggregate* corresponds to the aggregate root and can contain other dependent entities. In Fig. 2, *Tournament* (root) aggregate contains *TournamentCreator*.

2.2.2. Define microservices coordination

The coordination between microservices is strongly dependent on the transactional model. The simulator supports orchestrated coordination [6] where the TCC transactional model is applied. Fig. 3 exemplifies the coordination of microservices associated with

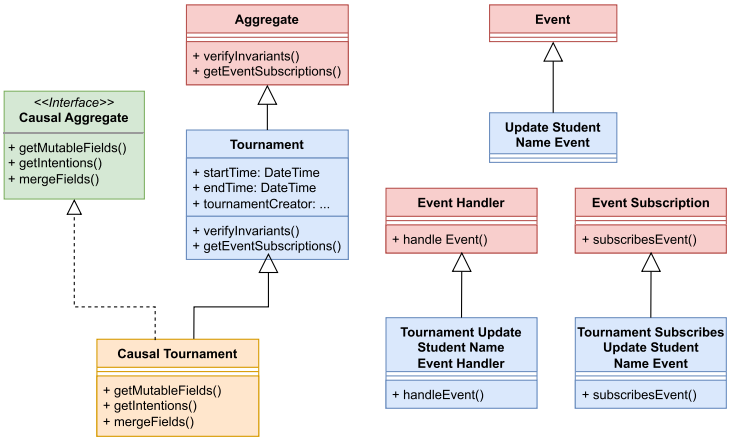


Fig. 2. Domain Model Extension.

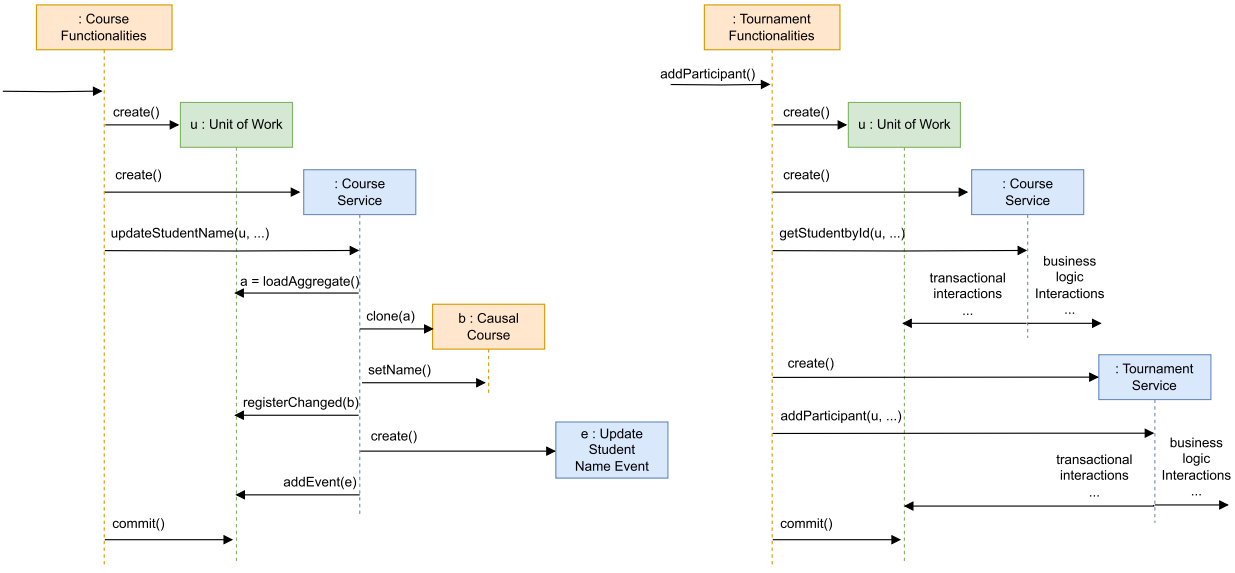


Fig. 3. Microservices Coordination.

the two functionalities described in [3]. On the left, it is a detailed description of a coordination that invokes a single microservice and emits an event. On the right, two microservices are coordinated. *Unit of Work* is responsible for supporting TCC. The functionality classes are part of the *coordination* package.

Fig. 4 exemplifies the implementation of event handlers that are triggered every second. Event handling logic (*Event Application Service*) is reused by all event handlings. It is only necessary to extend from *Event Handler* and define the event processing in its own TCC transaction, e.g. *processUpdateStudentNameEvent()*, which is implemented as a coordination.

3. Illustrative examples

An extensive example is implemented in package *quizzes* and its tests in *test/functionality*, as described in [3]. Next, we present the steps required to define the microservices model and design their tests.

3.1. Microservices modeling

Each microservice is modeled as an aggregate. The first step is to define the aggregates. The simulator uses Spring-Boot and JPA, so the domain entities definition uses the JPA notation. The code below shows the definition of `Tournament` aggregate as in Fig. 2. It has a root entity (`Tournament`) and two other internal entities (`TournamentCourse` and `Creator`).

```
@Entity
public abstract class Tournament extends Aggregate {
```

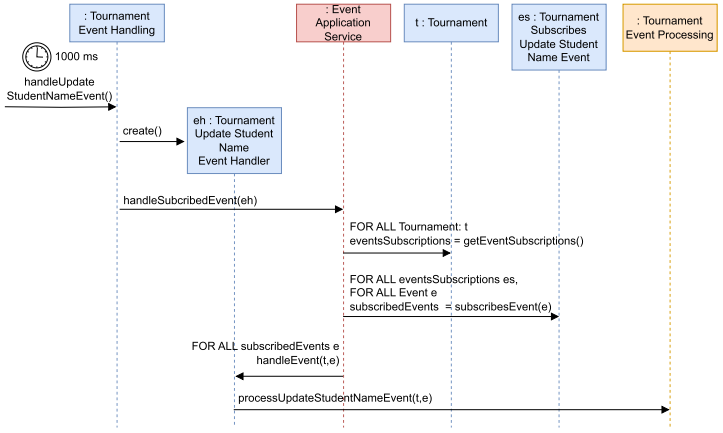


Fig. 4. Event Handling: Update Student Name Event.

```
    private LocalDateTime startTime;
    private LocalDateTime endTime;
    @OneToOne
    private TournamentCourse tournamentCourse;
    @OneToOne
    private Creator creator;
    ...
}
```

Then, the aggregate invariants are defined. In the example, since `Tournament` inherits from `Aggregate`, we override the method invoked by the `commit` method in the interactions of Fig. 3.

```
@Override
public void verifyInvariants() {
    if (!(this.startTime.isBefore(this.endTime))) {
            throw new TutorException(INVARIANT_BREAK, getAggregateId());
    }
}
```

Since the implementation is causal, it is necessary to enrich the `Tournament` class with the information required by the transactional causal consistency; therefore, it is necessary to override three methods, Fig. 2. The first override defines which attributes may need to be merged in the case of the occurrence of concurrent versions during commit. The second method defines in which situations the merge is possible, for instance, if one of the aggregates to merge changed the `startTime` attribute and the other the `endTime` attribute, it means the intentions of the users may not be preserved because each of them has a different intention about the tournament total duration time. The third method actually merges the data. Note that in the example, if both aggregates changed the `startTime` attribute, the last transaction would abort and the merge would never occur.

```
@Entity
public class CausalTournament extends Tournament implements CausalAggregate {
    ...
    @Override
    public Set<String> getMutableFields()  {
        return Set.of("startTime", "endTime", "tournamentCourse", "creator");
    }
    @Override
    public Set<String[]> getIntentions() {
        return Set.of(
                new String[]{"startTime", "endTime"});
    }
    @Override
    public Aggregate mergeFields(Set<String> toCommitVersionChangedFields,
                Aggregate committedVersion, Set<String> committedVersionChangedFields){
        ...
        CausalTournament committedTournament = (CausalTournament) committedVersion;
        CausalTournament mergedTournament = new CausalTournament(this);
        ...
        if (toCommitVersionChangedFields.contains("startTime")) {
            mergedTournament.setStartTime(getStartTime());
        } else {
            mergedTournament.setStartTime(committedTournament.getStartTime());
        }
```

```
        ...
        return mergedTournament;
    }
}
```

Events are persistent entities that support the communication between the upstream aggregates and the downstream ones, Fig. 2. They contain event-specific information. The `UpdateStudentNameEvent` contains a student identifier and the updated name.

```
@Entity
public class UpdateStudentNameEvent extends Event {
    private Integer studentAggregateId;
    private String updatedName;
    ...
}
```

Therefore, another additional definition required by the aggregate root entity is the subscription of events published by its upstream aggregates. This is done by overriding the `getEventSubscriptions` method in Fig. 2, which is used in the interactions in Fig. 4.

```
@Override
public Set<EventSubscription> getEventSubscriptions() {
    Set<EventSubscription> eventSubscriptions = new HashSet<>();
    eventSubscriptions.add(new TournamentSubscribesUpdateStudentName(this));
    return eventSubscriptions;
}
```

Subscriptions can depend on the state of the event, in addition to its type, which is defined by overriding a method of class *EventSubscription* (Fig. 2). This is illustrated by verifying that the update event for the student name corresponds to the creator of the tournament. Note that type verification is performed by invoking the super class *subscribesEvent* method.

```
@Overrride
public boolean subscribesEvent(Event event) {
    return super.subscribesEvent(event) && tournamentDto.getCreator().getAggregateId()
                                        .equals(updateStudentNameEvent.getStudentAggregateId());
}
```

In addition, the developer has to define an event handler (Fig. 2), which is used in the same interaction in Fig. 4. Note that the handler delegates the processing to a functionality that updates the name of the tournament creator, which is an attribute of the internal entity `Creator`.

```
@Override
public void handleEvent(Integer subscriberAggregateId, Event event) {
    this.tournamentEventProcessing.processUpdateStudentNameEvent(subscriberAggregateId,
}
```

Functionalities use the causal unit of work and invoke aggregate services, as illustrated by the `Course` functionality in Fig. 3.

```
public void updateStudentName(Integer courseAggregateId,
                              Integer userAggregateId, UserDto userDto) {
    CausalUnitOfWork unitOfWork = unitOfWorkService.createUnitOfWork(...);
    courseService.updateStudentName(courseAggregateId, userAggregateId,
                                    userDto.getName(), unitOfWork);
    unitOfWorkService.commit(unitOfWork);
}
```

The service executes an aggregate atomic transaction and interacts with the unit of work to register the changed objects and events to publish (Fig. 3).

```
@Transactional(isolation = Isolation.READ_COMMITTED)
public void updateStudentName(Integer courseAggregateId, Integer userAggregateId,
                                    String name, CausalUnitOfWork unitOfWork) {
    Course oldCourse = (Course) unitOfWorkService
                                .aggregateLoadAndRegisterRead(courseAggregateId,unitOfWork);
    Course newCourse = new CausalCourse((CausalCourse) oldCourse);
    newCourse.findStudent(userAggregateId).setName(name);
    unitOfWork.registerChanged(newCourse);
    unitOfWork.addEvent(new UpdateStudentNameEvent(courseAggregateId,userAggregateId, name));
}
```

3.2. Test design

To guarantee that the tests are deterministic, though simulating concurrent executions of functionalities, the simulator provides two techniques.

A functionality that writes an aggregate finishes and commits with version number n. When another functionality starts, it has version number $n+1$, which means that it reads the aggregate version created by the first functionality. This is a standard sequential execution. If we want to simulate a concurrent execution of these two functionalities, we can decrement the simulator kept version number by 1, between their executions. This results in the second functionality also starting with a version number of n like the first, instead of $n+1$. This ensures that the second functionality reads the same aggregate versions read by the first functionality. If the second functionality also performs a write on the same aggregate, upon committing, it detects the aggregate version written by the first functionality as concurrent, initiating a merge operation between the two. We can decide which concurrent functionality does the merge by running it after the version decrement, after the other has finished.

Concerning the second technique, event detection runs periodically at a given frequency, which depends on when the last event handling finished. However, during tests, it is sometimes useful to trigger the event detection at a precise moment in the test, for instance, when trying to test the concurrent execution of an event handling and a functionality. To achieve this, we can block periodic processing of events and decide when to explicitly trigger the detection and processing of a specific event type for an aggregate.

The following example illustrates the use of both techniques.

```
def 'concurrent add student as tournament participant and update name in course' () {
    given: 'student is added to tournament'
    tournamentFunctionalities.addParticipant(tournamentAggregateId, userAggregateId)
    and: 'the version number is decreased to simulate concurrency'
    versionService.decrementVersionNumber()
    and: 'student name is updated and the commit does not require merge'
    def updateNameDto = new UserDto()
    updateNameDto.setName(UPDATED_NAME)
    courseFunctionalities.updateStudentName(courseAggregateId,userAggregateId,updateNameDto)

    when: 'update name event is processed and participant is updated in tournament'
    tournamentEventHandling.handleUpdateStudentNameEvent();

    then: 'the name is updated in course execution'
    def course = courseFunctionalities.getCourseByAggregateId(courseAggregateId
    course.getStudents().find{it.aggregateId == userAggregateId}.name == UPDATED_NAME
    and: 'the name is updated in tournament'
    def tournament = tournamentFunctionalities.findTournament(tournamentAggregateId
    tournament.getParticipants().find{it.aggregateId==userAggregateId}.name==UPDATED_NAME
}
```

4. Impact

The existing systems that implement TCC [7,8] are technologically complex, making experimentation with complex business logic difficult. The *Transactional Causal Consistency* simulator allows testing the behavior of business logic rich microservices applications. It promotes the design of complex business logic transactional causal consistent microservices in a software developer-friendly environment. The simulator is a publicly available artifact that can be reused in other experiments.

5. Conclusions

The Microservices Simulator is an extensible object-oriented framework which decouples microservices business logic from transactional causal consistency and supports the rapid specification and prototyping of a microservice system. The simulator code and a large case study implementation are publicly available on Zenodo [9].

6. Future plans

Extend the simulator to also support the Saga transactional model [10,6], asynchronous communication, and choreography coordination.

CRediT authorship contribution statement

Pedro Pereira: Writing – original draft, Validation, Software, Investigation, Conceptualization. **António Rito Silva:** Validation, Supervision, Software, Methodology, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was partially supported by Fundação para a Ciência e Tecnologia (FCT) through projects UIDB/50021/2020 (DOI:10.54499/UIDB/50021/2020) and PTDC/CCI-COM/2156/2021 (DOI:10.54499/PTDC/CCI-COM/2156/2021).

References

[1] J. Thönes, Microservices, IEEE Softw. 32 (1) (2015) 116.
[2] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software, Addison Wesley, 2003.
[3] P. Pereira, A.R. Silva, Transactional causal consistent microservices simulator, in: M. Patiño-Martínez, J. Paulo (Eds.), Distributed Applications and Interoperable Systems, Springer Nature, Switzerland, Cham, 2023, pp. 57–73.
[4] D.D. Akkoorath, A.Z. Tomsic, M. Bravo, Z. Li, T. Crain, A. Bieniusa, N. Preguiça, M. Shapiro, Cure: strong semantics meets high availability and low latency, in: 2016 IEEE 36th International Conference on Distributed Computing Systems (ICDCS), 2016, pp. 405–414.
[5] M. Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, 2003.
[6] C. Richardson, Microservices Patterns, Manning Publications Co., 2019.
[7] C. Wu, V. Sreekanti, J.M. Hellerstein, Transactional causal consistency for serverless computing, in: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, SIGMOD '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 83–97.
[8] T. Lykhenko, R. Soares, L. Rodrigues, Faastcc: efficient transactional causal consistency for serverless computing, in: Proceedings of the 22nd International Middleware Conference, Middleware '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 159–171.
[9] P. Pereira, A.R. Silva, socialsoftware/business-logic-consistency-models: v1.1.4, https://doi.org/10.5281/zenodo.7854924, Jun. 2024.
[10] H. Garcia-Molina, K. Salem, Sagas, in: Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data, SIGMOD '87, Association for Computing Machinery, New York, NY, USA, 1987, pp. 249–259.